

Automated Rubric Assignment Evaluator

Software Requirements Specification (SRS)

Document Version: 1.0

Prepared for: Lab 1 – Requirements Engineering & UML Use-Case Modelling

Repository: --02Automated_rubric_assignment_evaluator

1. Introduction

1.1 Project Title

Automated Rubric Assignment Evaluator (ARAE).

1.2 Purpose

This document specifies the software requirements for the Automated Rubric Assignment Evaluator, a system that automatically evaluates student project submissions against a configurable rubric. It consolidates the functional and non-functional requirements, use cases, and interface behavior that the development team will implement and the test team will verify.

1.3 Scope

The system allows students to submit project work as a ZIP archive, automatically runs a configured test suite and syntax checks against the submission, computes a rubric-based score, and generates an evaluation report. Faculty Evaluators can review results, configure rubric criteria, assign peer reviews, and process re-evaluation requests. A System Administrator manages users, system configuration, and the evaluation infrastructure. The initial release targets a single-institution deployment handling submission spikes of up to 100 concurrent uploads.

1.4 Intended Users

- **Student** – submits project work and reviews feedback/scores.
- **Faculty Evaluator** – configures rubrics, reviews automated results, manages re-evaluation and peer review.
- **System Administrator** – manages accounts, configuration, and the underlying evaluation infrastructure.

2. Overall Description

2.1 Product Perspective

ARAE is a self-contained web-based evaluation platform. It is a new, independent product rather than a component of an existing system, although it is expected to integrate with an institution's existing submission/LMS workflow at the upload boundary. Submitted projects are processed in an isolated execution environment so that untrusted student code cannot affect the host system.

2.2 Product Functions (Summary)

- Accept and validate a student's project ZIP submission.
- Queue and execute a configured automated test suite against the submission.
- Calculate a rubric score breakdown from the test results.
- Generate and export a per-student evaluation report (PDF).
- Allow Faculty Evaluators to review, override, and justify scores, and manage re-evaluation requests.
- Allow Faculty Evaluators to configure rubric criteria and assign peer reviews.

- Allow the System Administrator to manage users, roles, and evaluation infrastructure.

2.3 User Classes and Characteristics

Actor	Interaction with the System
Student	Uploads project submissions and views evaluation results.
Faculty Evaluator	Reviews evaluation results, configures/uses evaluation criteria, and can review or request re-evaluation.
System Administrator	Manages users, system configuration, and evaluation infrastructure.

2.4 Assumptions and Constraints

- Submissions are accepted only as ZIP archives; other archive formats are out of scope for this release.
- Automated tests must run in an isolated execution environment for security and reproducibility.
- Peak load is assumed to occur near submission deadlines (up to 100 concurrent submissions).
- The system must not silently lose evaluation or grading data, given the high-stakes academic context.

3. Functional Requirements

The functional requirements below are reproduced from the project's authoritative requirements table (Requirements_Table.xlsx), which is the source of record for requirement wording, priority, and acceptance criteria.

ID	Pri.	Requirement	Acceptance Criteria
FR-001	High	The system shall automatically queue uploaded student project ZIP files, execute pre-configured unit test suites, and generate an itemized rubric score breakdown.	Pass: queues a valid ZIP, executes configured tests, and displays the rubric breakdown with test results within 10 seconds of submission. Fail: not queued, tests not executed, or breakdown missing/incorrect.
FR-002	High	The system shall allow a student to upload a project ZIP file for evaluation.	Pass: a valid ZIP uploads successfully and the system displays a unique submission ID and status. Fail: an invalid file is accepted, or no submission record is generated.
FR-003	High	The system shall execute the configured syntax checks and unit test cases against each submitted project.	Pass: all configured checks/tests execute and each result is recorded. Fail: any configured test is skipped without being reported.
FR-004	High	The system shall calculate and display scores for each rubric criterion based on the test results.	Pass: every rubric criterion receives a score and the displayed total matches the individual scores. Fail: a criterion has no score, or the total is incorrect.
FR-005	Medium	The system shall generate and export a per-student score report (PDF) summarizing rubric criterion scores, automated test results, and peer-review feedback.	Pass: a downloadable PDF generates within 15 seconds of request and contains all rubric line items, test outcomes, and peer comments. Fail: a section is missing or generation fails.

4. Non-Functional Requirements

As with the functional requirements, the non-functional requirements below are reproduced verbatim from Requirements_Table.xlsx.

ID	Pri.	Category	Requirement	Acceptance Criteria
NFR-001	High	Performance / Capacity	The system shall handle up to 100 concurrent project submissions without dropping queue items or exceeding a 1.5 GB memory footprint per worker process.	Pass: benchmarking confirms target latency/stability under 100 concurrent submissions, zero dropped items, memory under 1.5 GB. Fail: any submission dropped or memory ceiling exceeded.
NFR-002	High	Availability / Data Integrity	The system shall maintain at least 99.5% uptime during active submission and grading windows and guarantee no submission or grading data is lost if a service instance restarts.	Pass: uptime monitoring shows $\geq 99.5\%$ availability over the submission window and a simulated mid-restart test shows zero data loss. Fail: downtime exceeds the 0.5% error budget, or any in-flight submission/grade is lost on restart.

4.1 Additional Quality Behavior Evidenced by Completed Design Artifacts

The following behaviors are referenced by the completed use-case flow and are recorded here as design intent supporting NFR-001/NFR-002, pending formal requirement IDs:

- The rubric breakdown is rendered to the Student and Faculty Evaluator within 10 seconds (performance).
- Faculty score overrides are timestamped and the original automated score is retained in an audit trail (auditability).
- If evaluation fails, no incomplete score is saved; the submission is flagged for manual review (reliability).

5. Use Cases

5.1 Actors

Actor	Description
Student	Uploads project submissions and views evaluation results.
Faculty Evaluator	Reviews evaluation results, configures/uses evaluation criteria, reviews/requests re-evaluation.
System Administrator	Manages users, system configuration, and evaluation infrastructure.

5.2 Use-Case Diagram

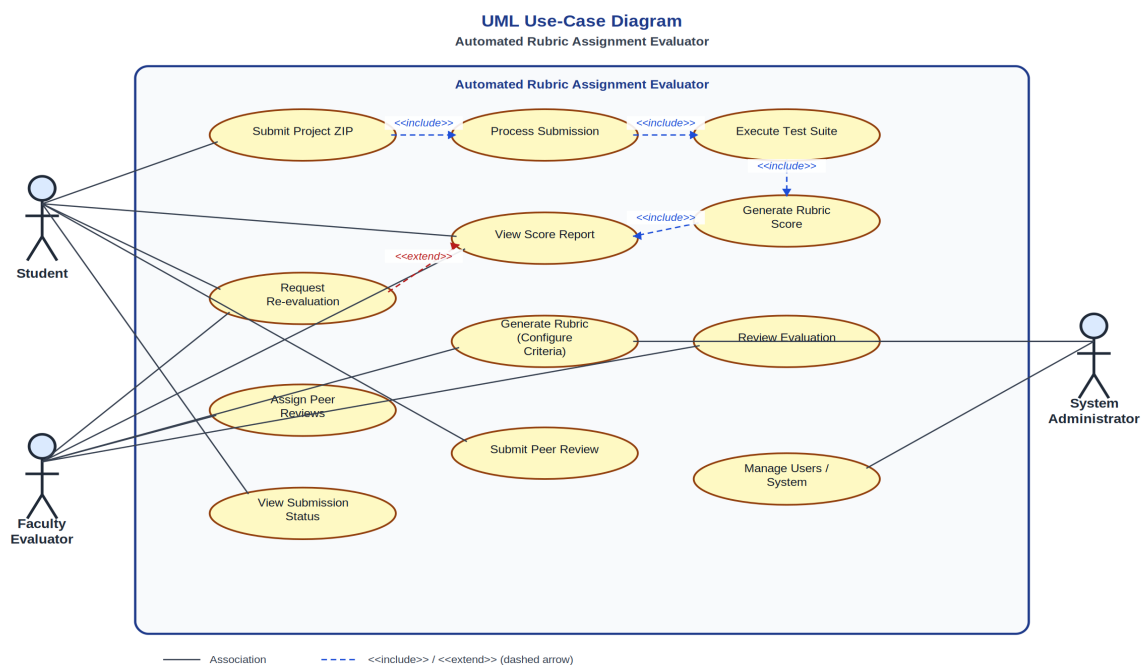


Figure 1: UML Use-Case Diagram – Automated Rubric Assignment Evaluator

5.3 Include / Extend Relationships

Relationship	Purpose
<>	Submit Project ZIP includes Process Submission.
<>	Process Submission includes Execute Test Suite.
<>	Execute Test Suite includes Generate Rubric Score.
<>	Request Re-evaluation extends View Score Report.

5.4 Use-Case Flow – Evaluate Project Submission

Primary Actor: Student **Supporting Actors:** Faculty Evaluator, System Administrator

Goal: Evaluate a submitted project using the defined rubric and generate an evaluation result.

Preconditions

- The student has a project submission ready.
- The evaluation system and rubric are available.

Postconditions

- The submission is evaluated.
- A rubric score is calculated.
- An evaluation report is generated.

Main Success Scenario

- 1 Student uploads the project submission in ZIP format.
- 2 The system receives and validates the submission.
- 3 The system executes the required test suite on the submission.
- 4 The system evaluates the submission against the defined rubric.
- 5 The system calculates the rubric score based on the evaluation results.
- 6 The system generates an evaluation report containing the results and score.
- 7 Faculty Evaluator reviews the evaluation result and report.
- 8 The system stores the evaluation result for future reference.

Alternate / Exception Flow

- **A1:** If the uploaded ZIP file is invalid or incomplete, the system rejects the submission and asks the Student to upload it again.
- **A2:** If a test cannot be executed successfully, the system records the failure and includes it in the evaluation result.

- **A3:** If the Faculty Evaluator identifies an issue with the evaluation, the submission can be reviewed or sent for re-evaluation.
- **A4:** If re-evaluation is requested, the system performs the evaluation again and generates an updated result.

6. External Interfaces

6.1 User Interfaces

- Student dashboard: submission upload form, submission status, score report view.
- Faculty dashboard: rubric configuration screen, evaluation review queue, peer-review assignment screen.
- Administrator console: user/role management, infrastructure and configuration settings.

6.2 Software Interfaces

- File upload interface accepting ZIP archives up to a configured size limit.
- Isolated test-execution environment (sandboxed runner) invoked per submission.
- PDF report generation service for exporting per-student score reports.

6.3 Communication Interfaces

- HTTPS for all client-server communication.
- Internal queue/messaging between the submission intake service and the test-execution workers.

7. Acceptance Criteria

Each functional and non-functional requirement carries its own pass/fail acceptance criteria, reproduced in Sections 3 and 4. At the system level, the release is considered acceptance-ready when:

- All FR-001 to FR-005 acceptance criteria pass under normal and peak-load conditions.
- NFR-001 and NFR-002 are verified by benchmarking and restart/failover testing.
- The use-case flow in Section 5.4, including its alternate/exception paths, executes correctly end to end.
- No in-flight submission or grade is lost during a simulated service restart.